

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



ĐỒ ÁN 1

Đề tài:

AI Agent hỗ trợ học Cấu trúc dữ liệu và Giải thuật

Mã học phần : IT5417

Mã lớp : 766770

GVHD : TS. Phạm Đăng Hải

Sinh viên : Bùi Anh Đức

MSSV : 20241582E

Hà Nội, Ngày 16 tháng 7 năm 2026

Mục lục

Nội dung

Chương I:	Giới thiệu.....	4
Chương II:	Cơ sở lý thuyết	4
1.	Phương pháp Socratic trong dạy học thuật toán.....	4
2.	LLM Agent và workflow có trạng thái	5
3.	Kiến trúc multi-agent.....	5
4.	Vai trò của testcase trong học thuật toán.....	5
Chương III:	Mục tiêu	6
1.	Mục tiêu tổng quát.....	6
2.	Mục tiêu cụ thể	6
2.1.	Điều phối hội thoại bằng LangGraph.....	6
2.2.	Ứng dụng LLM trong các tác vụ ngữ cảnh.....	6
2.3.	Xây dựng Quality Layer.....	6
2.4.	Cung cấp giao diện tương tác.....	6
Chương IV:	Công nghệ sử dụng	7
1.	Python cho core agent	7
2.	LLM provider cho năng lực hiểu ngôn ngữ	7
3.	LangGraph cho workflow orchestration.....	7
4.	Streamlit cho giao diện tương tác.....	8
5.	Quality Layer và sandbox execution	8
6.	Kiểm thử và đánh giá thành phần.....	8
Chương V:	Kiến trúc hệ thống	9
1.	Tổng quan kiến trúc.....	9

2. Tutor Orchestration Layer	9
3. Minh họa node LangGraph trong code.....	10
4. Learning Modules.....	10
5. Quality Layer.....	11
6. State & Memory Layer.....	11
7. Agent Trace và Observability.....	11
8. Thiết kế dữ liệu trạng thái	12
Chương VI: Kết luận.....	13
1. Những kết quả đã đạt được.....	13
2. Những hạn chế còn tồn tại.....	13
3. Hướng phát triển tiếp theo.....	14

TÓM TẮT— Báo cáo trình bày hệ thống DSA Socratic Agent, một LLM-powered multi-agent tutor hỗ trợ học Cấu trúc dữ liệu và Giải thuật. Hệ thống kết hợp mô hình ngôn ngữ lớn, LangGraph, Streamlit và các agent kiểm soát chất lượng để dẫn dắt người học theo phương pháp Socratic. Thay vì cung cấp lời giải trực tiếp, hệ thống phân tích ngữ cảnh, duy trì trạng thái học tập, sinh gợi ý theo mức độ, tạo testcase, kiểm tra code và đánh giá phản hồi trước khi gửi tới người học.

Chương I: Giới thiệu

Cấu trúc dữ liệu và Giải thuật là nhóm kiến thức nền tảng trong khoa học máy tính, nhưng quá trình học thường đòi hỏi nhiều hơn việc ghi nhớ thuật toán. Người học cần biết phân tích đề bài, xác định ràng buộc, lựa chọn cấu trúc dữ liệu phù hợp, kiểm tra edge case và đánh giá độ phức tạp. Đây là những kỹ năng khó hình thành nếu chỉ nhận lời giải hoàn chỉnh.

DSA Socratic Agent được thiết kế như một trợ giảng AI định hướng tư duy. Hệ thống sử dụng LLM để hiểu ngữ cảnh và sinh phản hồi tự nhiên, đồng thời sử dụng workflow agentic để chia quá trình xử lý thành các bước có cấu trúc. Cách tiếp cận này giúp kết hợp sự linh hoạt của LLM với khả năng kiểm soát của các agent chuyên trách.

Trọng tâm của hệ thống là kiến trúc multi-agent: AI Tutor Agent giữ vai trò điều phối, các learning modules xử lý nhiệm vụ chuyên môn, còn quality agents kiểm tra testcase, kết quả chạy code và mức độ phù hợp sự phạm của phản hồi.

Chương II: Cơ sở lý thuyết

1. Phương pháp Socratic trong dạy học thuật toán

Phương pháp Socratic là cách dạy học thông qua câu hỏi gợi mở thay vì truyền đạt trực tiếp đáp án. Trong bối cảnh học thuật toán, phương pháp này đặc biệt phù hợp vì mục tiêu không chỉ là biết một lời giải, mà là hiểu vì sao lời giải đó đúng, khi nào áp dụng được và giới hạn của nó nằm ở đâu.

Một phản hồi Socratic tốt thường bắt đầu từ việc yêu cầu người học mô tả lại input, output, ràng buộc và ví dụ nhỏ. Sau đó hệ thống mới dẫn dắt tới việc xác định trạng thái cần lưu,

thao tác cập nhật, invariant hoặc điều kiện dừng. Cách làm này giúp người học tự hình thành chuỗi suy luận thay vì sao chép lời giải.

Trong hệ thống này, Socratic tutoring được hiện thực hóa bằng `hint_level`, `policy` không đưa full solution và Pedagogy Critic Agent. Hint level giúp mức độ hỗ trợ tăng dần; `policy` giữ phản hồi không quá trực tiếp; còn Pedagogy Critic kiểm tra phản hồi cuối trước khi gửi cho người học.

2. LLM Agent và workflow có trạng thái

LLM có khả năng sinh ngôn ngữ tự nhiên và hiểu ngữ cảnh tốt, nhưng nếu chỉ gọi LLM trực tiếp cho mỗi câu hỏi thì hệ thống khó kiểm soát hành vi, khó kiểm thử và dễ trả lời không nhất quán. Vì vậy, project sử dụng LLM như một thành phần trong workflow có trạng thái thay vì xem LLM là toàn bộ hệ thống.

Một agent có trạng thái cần biết người học đang làm bài nào, đã xin bao nhiêu lần gợi ý, đã gửi code hay chưa, testcase nào từng được sinh và lần validation gần nhất cho kết quả gì. Các thông tin này được lưu trong `LearningState` để phản hồi ở lượt sau có tính liên tục.

3. Kiến trúc multi-agent

Kiến trúc multi-agent chia hệ thống thành nhiều thành phần chuyên trách. Thay vì một mô hình duy nhất vừa phân loại đề, vừa sinh testcase, vừa kiểm code, vừa đánh giá phản hồi, mỗi agent đảm nhiệm một vai trò rõ ràng. Cách tách này giúp giảm độ phức tạp trong prompt, dễ quan sát lỗi và dễ thay thế từng thành phần.

Trong DSA Socratic Agent, AI Tutor Agent là agent điều phối trung tâm. Các agent còn lại gồm Intent Detector, Problem Classifier, Hint Generator, Testcase Generator, Code Execution Validator, Pedagogy Critic và Session Title Agent. Mỗi agent có input/output có cấu trúc để workflow có thể kết hợp kết quả một cách ổn định.

4. Vai trò của testcase trong học thuật toán

Testcase không chỉ dùng để chấm đúng sai. Trong quá trình học thuật toán, testcase là công cụ giúp người học kiểm tra giả định, phát hiện edge case và trace trạng thái qua từng bước. Một testcase nhỏ nhưng được chọn đúng có thể làm lộ lỗi tư duy rõ hơn nhiều so với việc đọc lời giải hoàn chỉnh.

Vì vậy hệ thống phân biệt validation testcase và trace-only testcase. Validation testcase có expected output để chạy tự động. Trace-only testcase dùng khi expected output chưa đủ chắc chắn hoặc khi mục tiêu chính là hướng dẫn người học suy luận bằng tay.

Chương III: Mục tiêu

1. Mục tiêu tổng quát

Xây dựng một hệ thống AI Tutor hỗ trợ học Cấu trúc dữ liệu và Giải thuật bằng phương pháp Socratic, có khả năng hiểu hội thoại, duy trì trạng thái học tập, điều phối nhiều agent chuyên trách và kiểm soát chất lượng phản hồi.

2. Mục tiêu cụ thể

2.1. Điều phối hội thoại bằng LangGraph

Workflow của hệ thống được mô hình hóa bằng LangGraph với các node độc lập theo intent. Việc tách node giúp pipeline rõ ràng, dễ mở rộng, dễ kiểm thử và dễ quan sát trong quá trình vận hành.

2.2. Ứng dụng LLM trong các tác vụ ngữ cảnh

LLM được sử dụng cho intent detection, problem classification, hint generation, testcase generation, pedagogy review và final response generation. Các guideline nội bộ chỉ đóng vai trò hỗ trợ ngữ cảnh cho prompt, không thay thế khả năng sinh phản hồi của LLM.

2.3. Xây dựng Quality Layer

Quality Layer được thiết kế để tăng độ tin cậy của hệ thống. Testcase Generator Agent tạo case kiểm thử và case trace; Code Execution Validator Agent kiểm tra code khi có expected output; Pedagogy Critic Agent đánh giá phản hồi theo chính sách Socratic.

2.4. Cung cấp giao diện tương tác

Giao diện Streamlit cung cấp môi trường chat trực quan, hỗ trợ nhiều phiên học, hiển thị trạng thái học tập và Agent Trace. Giao diện đóng vai trò lớp trình diễn và tương tác, trong khi logic chính nằm ở core agent.

Chương IV: Công nghệ sử dụng

1. Python cho core agent

Python được sử dụng làm ngôn ngữ chính cho phần lõi của hệ thống. Các thành phần như AI Tutor Agent, LangGraph workflow, schema dữ liệu, testcase generator, validator và pedagogy review đều được tổ chức trong package Python. Việc sử dụng Python giúp hệ thống dễ tích hợp với thư viện AI, dễ viết prototype và thuận tiện cho kiểm thử tự động.

Trong project, các cấu trúc dữ liệu quan trọng được định nghĩa bằng dataclass. Cách tiếp cận này giúp biểu diễn rõ các object như LearningState, AgentTurn, Classification, TestSuite, ValidationResult và PedagogyReview. Nhờ đó, output giữa các agent không chỉ là text tự do mà có cấu trúc rõ ràng để workflow có thể xử lý tiếp.

2. LLM provider cho năng lực hiểu ngôn ngữ

Hệ thống sử dụng LLM provider để xử lý những tác vụ cần hiểu ngôn ngữ tự nhiên và ngữ cảnh học tập. Các tác vụ này gồm nhận diện intent, phân loại dạng bài, sinh gợi ý, sinh testcase, đánh giá phản hồi sư phạm và tạo phản hồi cuối cho người học.

Việc dùng LLM giúp hệ thống linh hoạt hơn so với classifier hoặc hint engine thuần rule-based. Ví dụ, cùng một bài toán quy hoạch động có thể được mô tả bằng nhiều cách khác nhau; LLM có khả năng nhận diện tín hiệu ngữ nghĩa như trạng thái, lựa chọn tối ưu, quan hệ truy hồi hoặc điều kiện biên mà không cần phụ thuộc hoàn toàn vào keyword cố định.

3. LangGraph cho workflow orchestration

LangGraph được sử dụng để mô hình hóa quá trình xử lý hội thoại thành một đồ thị có trạng thái. Mỗi node trong graph tương ứng với một bước hoặc một hành vi chính của agent, chẳng hạn `detect_intent`, `submit_problem`, `request_hint`, `submit_code`, `direct_solution_guard`, `ask_theory` và `finalize`.

So với cách viết một hàm xử lý duy nhất, LangGraph giúp luồng xử lý dễ quan sát và dễ mở rộng hơn. Khi cần thêm một agent mới, hệ thống có thể bổ sung node hoặc cạnh điều kiện mà không phải viết lại toàn bộ orchestrator. Điều này đặc biệt phù hợp với kiến trúc multi-agent, nơi nhiều thành phần chuyên trách cần phối hợp theo từng ngữ cảnh.

LangGraph cũng giúp báo cáo và demo rõ ràng hơn vì có thể giải thích hệ thống như một pipeline: người học gửi input, graph phát hiện intent, route sang node phù hợp, gọi các module/agent cần thiết, sau đó quay về finalize để trả phản hồi cuối.

4. Streamlit cho giao diện tương tác

Streamlit được sử dụng để xây dựng giao diện chat cho hệ thống. Giao diện gồm danh sách phiên chat, vùng hội thoại, ô nhập và panel trạng thái. Công nghệ này phù hợp vì cho phép xây dựng nhanh một giao diện tương tác phục vụ demo, đồng thời vẫn đủ linh hoạt để hiển thị state, testcase, validation result và Agent Trace.

Trong hệ thống AI tutor, giao diện không chỉ là nơi nhập câu hỏi. UI còn giúp người học theo dõi tiến trình học, xem gợi ý theo từng lượt, quan sát các testcase gợi ý và biết hệ thống đang phân loại bài toán theo hướng nào. Vì vậy Streamlit đóng vai trò quan trọng trong việc biến core agent thành một trải nghiệm học tập có thể sử dụng được.

5. Quality Layer và sandbox execution

Quality Layer gồm các thành phần hỗ trợ kiểm soát chất lượng phản hồi. Testcase Generator Agent tạo testcase để người học tự kiểm hoặc để validator chạy tự động. Code Execution Validator Agent chạy code Python trên testcase có expected output và trả về trạng thái chuẩn hóa như PASSED, FAILED, RUNTIME_ERROR, TIMEOUT hoặc SKIPPED.

Sandbox execution trong phiên bản hiện tại tập trung vào Python. Việc giới hạn ngôn ngữ và thời gian chạy giúp giảm rủi ro khi thực thi code do người học gửi. Kết quả chạy code không được trả thẳng dưới dạng log kỹ thuật, mà được Tutor Agent chuyển hóa thành gợi ý học tập như case nào cần trace, biến nào cần kiểm tra hoặc điều kiện biên nào có thể gây sai.

6. Kiểm thử và đánh giá thành phần

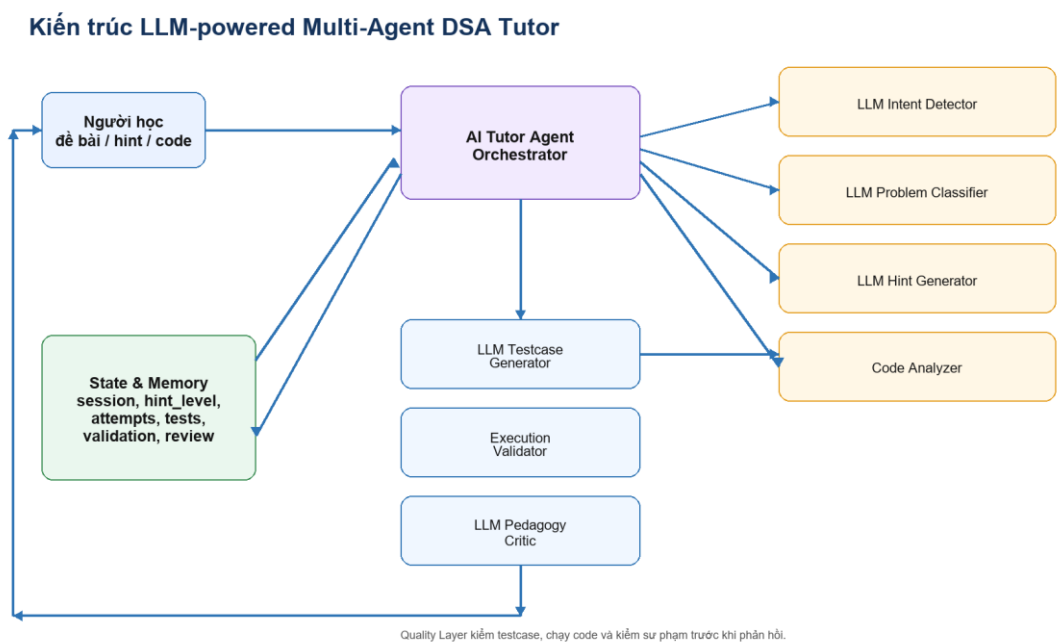
Hệ thống sử dụng unittest để kiểm tra các thành phần cốt lõi như intent handling, schema testcase, validation result, pedagogy review và các nhánh xử lý chính của agent. Kiểm thử giúp đảm bảo các agent trả output có cấu trúc ổn định và workflow có thể hoạt động nhất quán.

Bên cạnh unit test, hệ thống còn được thiết kế để đánh giá bằng các kịch bản hội thoại. Các kịch bản này bao gồm gửi đề bài, xin gợi ý nhiều lần, gửi code sai edge case và yêu cầu lời giải trực tiếp. Đây là cách kiểm tra phù hợp với một hệ thống tutor vì chất lượng không chỉ nằm ở đúng/sai kỹ thuật mà còn ở cách phản hồi dẫn dắt người học.

Chương V: Kiến trúc hệ thống

1. Tổng quan kiến trúc

Hệ thống gồm năm lớp chính: User Interface, Tutor Orchestration Layer, Learning Modules, Quality Layer và State & Memory Layer. Các lớp này tách biệt trách nhiệm giữa giao diện, điều phối, xử lý chuyên môn, kiểm soát chất lượng và lưu trạng thái học tập.

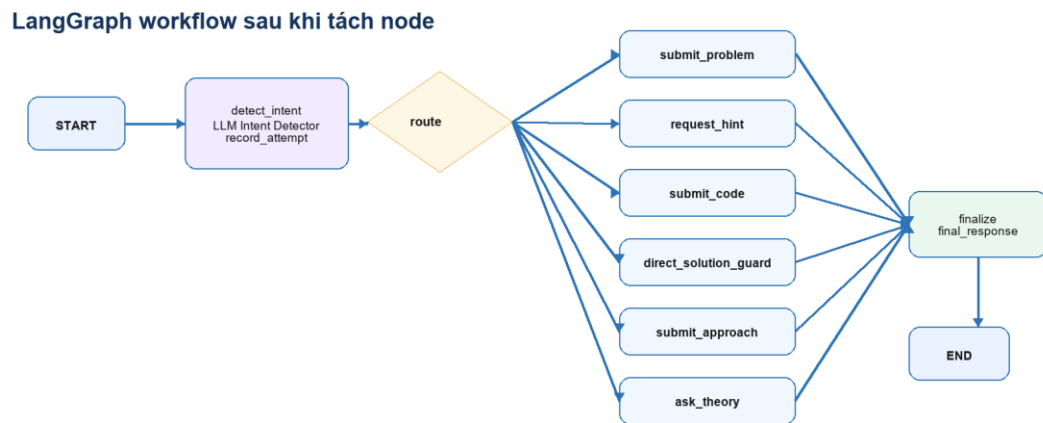


Hình 1: Tổng quan kiến trúc hệ thống multi-agent

2. Tutor Orchestration Layer

AI Tutor Agent là thành phần điều phối trung tâm. Agent nhận message từ người học, gọi LLM để xác định intent, cập nhật LearningState, định tuyến tới module hoặc agent phù hợp, sau đó tổng hợp kết quả thành phản hồi cuối.

Workflow LangGraph gồm các node `detect_intent`, `submit_problem`, `request_hint`, `submit_code`, `direct_solution_guard`, `submit_approach`, `ask_theory` và `finalize`. Mỗi node xử lý một loại hành động cụ thể và trả output có cấu trúc để các bước sau sử dụng.



Mỗi intent đi qua một node riêng nên có thể mở rộng, debug và trình bày rõ hơn trong báo cáo đồ án.

Hình 2: Workflow LangGraph tách node theo intent

3. Minh họa node LangGraph trong code

Đoạn code dưới đây thể hiện cách workflow được tách thành các node độc lập. Mỗi node đại diện cho một hành vi chính của tutor agent, còn conditional edge đảm nhiệm việc định tuyến theo intent.

```
src/dsa_agent/graph_workflow.py

01 graph = StateGraph(TutorGraphState)
02 graph.add_node("detect_intent", self._detect_intent_node)
03 graph.add_node("submit_problem", self._submit_problem_node)
04 graph.add_node("request_hint", self._request_hint_node)
05 graph.add_node("submit_code", self._submit_code_node)
06 graph.add_node("direct_solution_guard", self._direct_solution_guard_node)
07 graph.add_node("submit_approach", self._submit_approach_node)
08 graph.add_node("ask_theory", self._ask_theory_node)
09 graph.add_node("finalize", self._finalize_node)
10
11 graph.add_edge(START, "detect_intent")
12 graph.add_conditional_edges("detect_intent", self._route_intent, {...})
13 graph.add_edge("finalize", END)
14
Trích đoạn code minh họa từ project
```

Hình 3: Trích đoạn khai báo các node LangGraph trong project

4. Learning Modules

Learning Modules cung cấp năng lực chuyên môn cho quá trình dạy học. Problem Classifier nhận diện chủ đề và pattern thuật toán; Hint Generator sinh gợi ý theo hint level;

Code Analyzer phân tích logic, độ phức tạp và edge case; Knowledge Context cung cấp taxonomy và rubric nội bộ cho prompt.

Các module này không hoạt động độc lập như các chatbot nhỏ riêng lẻ, mà được Tutor Agent gọi theo ngữ cảnh. Kết quả của module được lưu vào state hoặc truyền sang Quality Layer để kiểm tra thêm.

5. Quality Layer

Quality Layer gồm ba agent chính. Testcase Generator Agent sinh TestSuite với các case basic, edge, adversarial hoặc stress. Code Execution Validator Agent chạy code Python trên testcase có expected output. Pedagogy Critic Agent đánh giá phản hồi nháp trước khi gửi cho người học.

Testcase được chia thành hai nhóm: validation testcase và trace-only testcase. Validation testcase có expected output và dùng để chạy kiểm tra tự động. Trace-only testcase dùng để hướng dẫn người học tự kiểm tra logic khi đề bài chưa đủ rõ để tạo oracle tin cậy.

Pedagogy Critic kiểm tra các rủi ro như lộ full solution, phản hồi quá trực tiếp, không phù hợp hint level hoặc đưa quá nhiều gợi ý trong một lượt. Nếu phản hồi chưa đạt, hệ thống yêu cầu LLM viết lại trước khi gửi.

6. State & Memory Layer

LearningState lưu trữ session_id, current_problem, problem_type, concepts, hint_level, student_attempts, misconceptions, generated_tests, latest_validation, pedagogy_flags và next_action. Nhờ state, hệ thống có thể cá nhân hóa phản hồi theo tiến trình học thay vì xử lý từng câu hỏi độc lập.

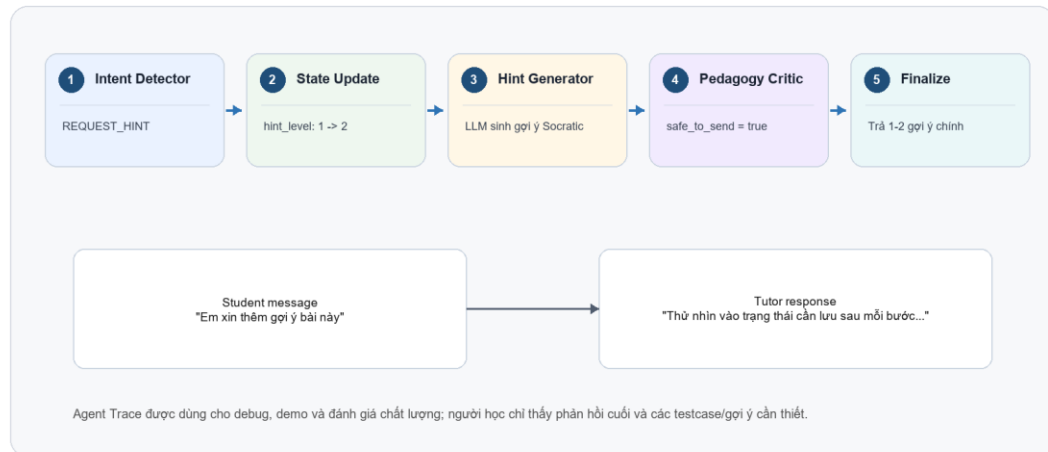
Session Title Agent sinh tiêu đề ngắn cho từng phiên chat dựa trên nội dung trao đổi ban đầu. Cách này giúp người dùng quản lý nhiều phiên học hiệu quả hơn và tránh hiển thị nguyên văn message dài ở danh sách phiên.

7. Agent Trace và Observability

Agent Trace ghi lại các bước workflow đã chạy trong một lượt hội thoại, bao gồm node name, trạng thái, summary và output rút gọn. Trace này phục vụ debug, demo và đánh giá hệ thống; nó không phải là chain-of-thought nội bộ của LLM.

Agent Trace cho một lượt hội thoại

Trace ghi lại các bước workflow đã chạy, không hiển thị chain-of-thought nội bộ của LLM.



Hình 3: Minh họa Agent Trace trong một lượt hội thoại

8. Thiết kế dữ liệu trạng thái

LearningState là lớp dữ liệu trung tâm của hệ thống. Thay vì chỉ lưu lịch sử chat dạng text, state lưu cả thông tin học tập như dạng bài, concepts, hint level, attempts, testcase history, validation result và pedagogy flags. Đây là cơ sở để phản hồi của agent có tính cá nhân hóa.

Nhờ LearningState, cùng một câu 'cho em thêm gợi ý' có thể tạo ra phản hồi khác nhau tùy vào bài toán hiện tại, số lần người học đã xin hint và kết quả validation gần nhất. Điều này giúp hệ thống khác với chatbot stateless thông thường.

```
src/dsa_agent/models.py

01 @dataclass
02 class LearningState:
03     session_id: str = field(default_factory=lambda: str(uuid4()))
04     current_problem: str | None = None
05     problem_type: str | None = None
06     concepts: list[str] = field(default_factory=list)
07     hint_level: int = 0
08     student_attempts: list[str] = field(default_factory=list)
09     misconceptions: list[str] = field(default_factory=list)
10     generated_tests: list[TestSuite] = field(default_factory=list)
11     latest_validation: ValidationResult | None = None
12     pedagogy_flags: list[PedagogyReview] = field(default_factory=list)
13     next_action: str = "classify_or_ask"
14 Trích đoạn code minh họa từ project
```

Hình 6: Trích đoạn schema LearningState

Chương VI: Kết luận

1. Những kết quả đã đạt được

Hệ thống đã xây dựng được kiến trúc tổng thể cho một AI tutor hỗ trợ học Cấu trúc dữ liệu và Giải thuật theo phương pháp Socratic. Thay vì chỉ là chatbot hỏi đáp, hệ thống có AI Tutor Agent điều phối trung tâm, LangGraph workflow để định tuyến hội thoại, LearningState để lưu trạng thái học tập và Quality Layer để kiểm soát phản hồi.

Một ưu điểm quan trọng của hệ thống là khả năng sử dụng LLM trong các bước cần hiểu ngữ cảnh. LLM được dùng để nhận diện intent, phân loại bài toán, sinh gợi ý, sinh testcase và đánh giá phản hồi sư phạm. Nhờ đó, hệ thống linh hoạt hơn so với cách tiếp cận rule-based thuần túy.

Hệ thống cũng đã có cơ chế gợi ý nhiều cấp độ thông qua hint_level. Khi người học xin thêm gợi ý, agent không lập tức đưa lời giải mà tăng dần mức hỗ trợ. Cách làm này phù hợp với mục tiêu học tập vì người học vẫn phải tự suy luận, tự trace và tự hoàn thiện lời giải.

Quality Layer là điểm mạnh khác của thiết kế. Testcase Generator Agent giúp tạo các case để kiểm tra giả định; Code Execution Validator Agent hỗ trợ phát hiện lỗi khi có expected output; Pedagogy Critic Agent kiểm tra phản hồi nháp trước khi gửi. Nhờ đó, hệ thống có thêm lớp kiểm soát thay vì phụ thuộc hoàn toàn vào một lần sinh phản hồi của LLM.

Về mặt triển khai, hệ thống đã có giao diện Streamlit, workflow LangGraph tách node rõ ràng, schema dữ liệu cho các output quan trọng và các hình minh họa kiến trúc/code trong báo cáo. Đây là nền tảng đủ rõ để tiếp tục phát triển thành một trợ giảng AI hoàn chỉnh hơn.

2. Những hạn chế còn tồn tại

Hạn chế đầu tiên là hệ thống chưa có cơ sở dữ liệu bền vững để lưu lịch sử học tập lâu dài. LearningState hiện phù hợp cho từng phiên tương tác, nhưng để sử dụng thực tế cần bổ sung database để lưu session, tiến độ học, misconceptions và lịch sử testcase theo từng người học.

Hạn chế thứ hai nằm ở sandbox execution. Phiên bản hiện tại tập trung vào Python và chỉ xử lý được các testcase có expected output rõ ràng. Với các bài toán có input/output phức

tập hoặc cần nhiều file, validator cần được mở rộng về môi trường chạy, giới hạn tài nguyên và cơ chế chuẩn hóa kết quả.

Hạn chế thứ ba là testcase generator vẫn phụ thuộc vào độ rõ của đề bài và chất lượng sinh của LLM. Trong một số trường hợp, testcase chỉ nên dùng để trace tay thay vì validation tự động. Nếu ép sinh expected output khi đề chưa đủ thông tin, hệ thống có thể tạo oracle sai.

Ngoài ra, hệ thống chưa có bộ benchmark lớn để đánh giá định lượng chất lượng phản hồi. Các tiêu chí như mức độ Socratic, khả năng tránh lộ lời giải, độ chính xác phân loại bài toán và hiệu quả phát hiện edge case cần được đo trên nhiều bài tập và nhiều kịch bản hội thoại hơn.

3. Hướng phát triển tiếp theo

Hướng phát triển đầu tiên là bổ sung database cho State & Memory Layer. Khi có lưu trữ bền vững, hệ thống có thể theo dõi tiến trình học của từng người dùng qua nhiều phiên, phát hiện misconception lặp lại và cá nhân hóa gợi ý tốt hơn.

Hướng phát triển thứ hai là nâng cấp sandbox execution. Hệ thống có thể sử dụng môi trường cách ly mạnh hơn, hỗ trợ nhiều ngôn ngữ lập trình hơn, kiểm soát bộ nhớ/thời gian chạy chặt chẽ hơn và chuẩn hóa input/output theo từng dạng bài.

Hướng phát triển thứ ba là xây dựng Evaluation Agent và benchmark đánh giá. Evaluation Agent có thể chấm phản hồi theo rubric như đúng hint level, không lộ full solution, có tận dụng testcase hay không và có đặt câu hỏi dẫn dắt phù hợp không.

Cuối cùng, hệ thống có thể mở rộng UI để hiển thị tiến độ học tập, thống kê dạng bài đã luyện, testcase thường sai và khuyến nghị chủ đề cần ôn lại. Khi kết hợp state dài hạn, evaluation và giao diện học tập, DSA Socratic Agent có thể trở thành một trợ giảng AI cá nhân hóa cho môn Cấu trúc dữ liệu và Giải thuật.